BreedVision AI: Multimodal Hybrid Pipeline of Cattle. Breeding of Buffalos with CNN-ONNX Inference, LLM Vision. Fallback, and Fuzzy Matching Knowledgeable of the Dataset.

Rohan Das
Dept. of Computer Science and Engineering
Presidency University, Bengaluru
itzrohan2486@gmail.com

Dr. Jayavadivel Ravi
Associate Professor
School of Computer Science and Engineering
Presidency University, Bengaluru
Faculty Guide

Amrutha S
Dept. of Computer Science and Engineering
Presidency University, Bengaluru
amrutha7768@gmail.com

Abstract*—Appropriate breeds of livestock must be used regarding accurate livestock breeding, genetics, and optimal dairy production programs in India. Morphological classification remains a professional exercise that is still quite cumbersome and can be subject to observer variations that range between 25 per cent in phenotypically similar breeds. BreedVision AI (BreedAI) is here to help with cattle and buffalo mobile and web platform for classification of cattle and buffalo breeds through a single image input. An effective hybrid artificial intelligence pipeline has been designed in our program to leverage three layers of inference that mutually reinforce each other. Layer 1: Microservice for CNN ONNX Runtime ONNXRuntime2025; Layer 2: Multimodal Large Language model ONNX Runtime ONNXruntime2025 Model Fallback LLM [7], [26]; Layer 3: Histogram matching of fuzzy label feature vectors of dataset-sensitized 32-bins reconciliation histogram vectors by Levenshtein distance [12], [14]. Our model leverages multi-factor confidence calibration based on maximum entropy probabilities in images dependent upon certain quality features of the images - brightness, contrast, Laplacian variance, sharpness, resolution, good faith, margin of confidence and entropy of prediction per request [9], [10]. React 18 + TypeScript represents a powerful single-page application framework that allows for Supabase Auth JWT session management technique along with Row-Level Security and real-time Postgres database with user-level history of classifications using subscriptions is used [24]. Additionally, a companion app for iOS and Android is built using React Native/Expo SDK 54 pipeline for the same pipeline on iOS and Android [19]. Experiments performed on a comparison between twenty Indian breeds of cattle and buffaloes have an average calibration error of 73.8% and CNN-path 98% model with an average latency of 187 ms due to the multi-layered fallback architecture.

***Index Terms*—Convolutional neural network, ONNX Runtime, classification of breeds. Large language model, multimodal vision, feature vector matching, confidence calibration, Supabase Edge Functions, Levenshtein distance, React TypeScript, react native, image quality measures, livestock. informatics, dairy management, Indian buffalo, Indian cattle.**

I. Introduction

India houses nearly 193 million kineen. 2.9 million buffaloes - making the largest herd of cattle across the globe. Such a source of wealth is used efficiently and based on appropriate information regarding it. breeding of which influences potential milk output, immunity from diseases, order of temperature acclimatization, and preservation. The method of identification, traditional and as practiced by qualified veterinarians, is the study of physical traits. However, this technique is inflexible, time-consuming, and prone to intra-rater error of up to. 25Tharparkar [2].

The problem lies in the rapid democratisation of the deep convolutional network and its democratisation. That is, it is new visual language model, thus creating a multimodal framework. classification process is trivial for breed recognition. The difficulties which cannot be overcome at this moment with existing prototypes [1], [4] include: (i) CNN classifiers have inherent weakness of the so-called closed-set assumption. in that they work with local breeds not in terms of training; (ii) field conditions. there exists a high dispersion of photometrics and, therefore, reduced confidence in images. without an easy-to-realize solution for the mentioned issue 2020calibration; and (iii) no livestock application that incorporates authentication,

All of the above issues are solved using the BreedVision AI system. The main innovations in the system are:

1) A comprehensive full-stack production system that is fully authenticated. equivalent to CNN, LLM and datasets running on one Supabase server. edgefunctions Deno Edge Function call 2026 2026edgefunctions. [22].
2) An innovative multi-factor confidence calibration method for item A. including image quality score, softmax margin, and prediction entropy. [9], [10], [11]. item A 32-binned RGB histogram-based feature vector pipeline that provides. The degree of similarity between

the image at a level in the dataset, and no second level. An embedding system [12], [14].
item Fuzzy match labeling by Levenshtein-distance, reconcile. when cannons items were labeled. [14], [17].
item A React Native / Expo SDK 54 mobile companion app. Same Supabase backend application [19], [21].

3) The item-reference-code is an open-source project. https://github.com/Rohan2486/CSE_40_UniversityProject [23].

II. Literature Review

A. Classical and Handcrafted-Feature Approaches

In the initial application of machine learning in breed classification, the indices used were derived from the body features and LBP of the image of the animal taken at specified postures. Competent performance was achieved using ensemble methods based on structured morphological information [2] which needed maintaining fixed camera distance and white background that cannot be achieved in the field. The color segmentation approach to coat pattern has been explored for uniform coat types; it is not generalizable to Indian phenotypes [1].

B. CNN Approaches for Livestock

There are several successful examples of lightweight deep learning models applied to breed identification of dairy cattle in India [4]. Machine Learning-based cattle breed classification in Kenya demonstrates high accuracy with ImageNet-trained models requiring only moderate finetuning for various regions [5]. Nevertheless, one major drawback of CNNs is that they suffer from the **closed set problem** which means that an unknown breed would be assigned to the breed class with the highest confidence in the model based on training breeds [1] [2].

C. LLM MultiModal Image Understanding as Fallback

Large language models trained on multimodal data can identify breeds in a zero-shot fashion. Google's Gemini supports multimodal image understanding through its JSON structured output, making it suitable for breed classification [26], [8]. Testing showed good results with respect to structured responses to multimodal input queries for LlamaIndex using Google Gemini [3]. GPT-4o-mini by OpenAI is an effective and cheap endpoint for vision tasks, with a rate of $0.001-$0.005 per image [7]. The unique feature of our solution is combining the two approaches intelligently.

D. Model Calibration

Modern deep learning models are universally overconfident: the ECE on benchmarks is above 0.05 unless post-hoc calibration is applied [9]. In face verification research, image quality is considered to be the primary source of confidence variations, and using sharpness based on Laplacian variance with brightness normalization gives the highest ECE decrease [10], [15]. Composite multi-factor scores have proven to provide more improvements than temperature scaling alone [11], [13].

E. Fuzzy Matching of Strings for Labelling

The Levenshtein Edit Distance is an example of a classical approach to calculating similarities between strings and is often applied to probability-based database matching [12],

F. BreedAI Positioning

To the best of my knowledge, there have been no efforts towards combining CNNs for inference, large language models for fallback, fuzzy matching of strings, generating histograms of feature vectors, Supabase user histories, and React Native application implementation into a unified livestock classification system. BreedAI fills this niche, offering both system design and a confidence scoring model trained on twenty Indian breeds.

III. Methodology

A. Classification Pipeline for Hybrid AI

The classification pipeline manages up to three levels of inference, which are executed in priority order depending on the value of the parameter `inferenceMode` [1], [2].

Fig. 1: Pipeline Architecture for Hybrid AI Classification: input images undergo inference using the CNN CNN main inference, LLM cascading for low confidence, multiple criteria calibration, Levenshtein fuzzy matching, and Supabase historical data storage. Timing markers indicated below right.

The time complexity is $\mathcal{O}(T_{\text{cnn}} + T_{\text{llm}} + T_{\text{cal}} + T_{\text{fuzzy}})$, where T_{cnn} (500–2000 ms), T_{llm} (2000–5000 ms, only if CNN fails), T_{cal} (50–100 ms), and T_{fuzzy} is $\mathcal{O}(n \cdot m \cdot k)$ [1]. Best case: 500–2000 ms. Worst case: 8000–12000 ms.

B. CNN-Based Image Classification

The CNN microservice employs either **MobileNetV2** or **EfficientNet-B0** as a lightweight base model pre-trained on the ImageNet dataset, with further fine-tuning performed on the Indian cattle and buffalo breeds. The classification network architecture is composed of Global Average Pooling, a Dense layer (256 neurons, activation function = ReLU), Dropout (dropout rate=0.3), and a softmax output for $N + 1$ breed classes plus an `unknown` class.

Image pre-processing follows the standard ImageNet normalisation pipeline:

$$\hat{x} = \frac{x/255 - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \quad \boldsymbol{\mu} = [0.485,\ 0.456,\ 0.406], \quad \boldsymbol{\sigma} = [0.229,\ 0.224,\ 0.225] \tag{1}$$

Following the forward pass in ONNX Runtime session, temperature scaling T is used to adjust logits before softmax transformation [9]:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{2}$$

Prediction entropy and softmax margin quantify uncertainty [9], [11]:

$$H_n = -\frac{\sum_i p_i \log p_i}{\log N} \tag{3}$$

$$M = p_{(1)} - p_{(2)} \tag{4}$$

CNN complexity: $\mathcal{O}(H \cdot W \cdot C \cdot K^2 \cdot F)$ per layer. Model size: 10–20 MB quantized; inference: 500–2000 ms CPU, 50–200 ms GPU [4], [2].

C. Multimodal LLM Vision Fallback

When CNN confidence falls below `CNN_MIN_CONFIDENCE` (default 45%), the edge function escalates to LLM inference. OpenAI GPT-4o-mini is the primary fallback; Google Gemini 2.5 Flash is the final fallback [3], [7], [8]. Both APIs receive a structured prompt constraining the response to JSON with fields: `type`, `breed`, `confidence` (0–1), `explanation`, `visible_traits`, and `uncertainty_factors`. Temperature is fixed at 0.3 [6]. API latency: OpenAI 2000–5000 ms; Gemini 1850–2900 ms (p95) [7], [8].

D. Multi-Factor Confidence Calibration

Raw model confidence is adjusted by a composite calibration formula [9], [10], [11]:

$$B = 0.72 \cdot P_{\text{cnn}} + 0.22 \cdot Q_{\text{img}} \tag{5}$$

$$C = \text{clip}\big(B - \alpha_w W - \alpha_m (0.18 - M)^+ - \alpha_e H_n,\ 0,\ 100\big) \tag{6}$$

where P_{cnn} is raw CNN confidence (0–100), Q_{img} is image quality score (0–100), W is the count of CV warnings, M is the softmax margin from Eq. (4), and H_n is normalised entropy from Eq. (3). Penalty coefficients $\alpha_w = 3$, $\alpha_m = 55$, $\alpha_e = 12$ were determined empirically [9]. $(\cdot)^+$ denotes $\max(\cdot, 0)$, applied only when $M < 0.18$.

The image quality score Q_{img} aggregates four photometric metrics [13], [15], [16]:

$$Q_{\text{img}} = 100 \cdot (0.30\, f_b + 0.25\, f_c + 0.30\, f_s + 0.15\, f_r) \tag{7}$$

where $f_b = 1 - |\mu_{\text{brightness}} - 128|/128$; $f_c = \min(1, \sigma_{\text{contrast}}/64)$; $f_s = \min(1, \text{Var}(\nabla^2 I)/60)$ (Laplacian sharpness); $f_r = \min(1, \min(W_{px}, H_{px})/1024)$ [15].

E. Dataset-Aware Fuzzy Label Matching

Levenshtein edit distance [12], [14], [17] reconciles predicted breed names to canonical dataset entries. For predicted string s_1 and candidate s_2:

$$\text{sim}(s_1, s_2) = 1 - \frac{\text{Lev}(s_1, s_2)}{\max(|s_1|, |s_2|)} \tag{8}$$

Matching proceeds: (1) exact match after Unicode normalisation and case-folding; (2) fuzzy match via Eq. (8) with threshold $\tau = 0.80$. Confidence multipliers: sim $\geq 0.95 \rightarrow 0.98$; sim $\geq 0.85 \rightarrow 0.90$; sim $\geq 0.80 \rightarrow 0.80$; no match $\rightarrow 0.70$ penalty. Complexity: $\mathcal{O}(n \cdot m \cdot k)$ [17].

F. Feature-Vector Construction and Cosine Similarity

32-bin RGB histograms form a 96-dimensional feature vector $\mathbf{v} \in \mathbb{R}^{96}$. Cosine similarity at inference time:

$$\text{sim}(\mathbf{v}_q, \mathbf{v}_r) = \frac{\mathbf{v}_q \cdot \mathbf{v}_r}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}_r\|} \tag{9}$$

Records with sim $\geq$ 0.85 are surfaced in `accuracyReports`. Feature vectors are pre-computed and stored as JSONB arrays in Supabase PostgreSQL [24].

IV. Implementation

A. Edge Function: classify-animal (Deno)

The classification orchestrator is a Supabase Edge Function written in TypeScript running on the Deno V8 runtime [24], [22]. It stores a cache of labels for each dataset stored in-process (TTL = 300 s).

Listing 1: Edge Function — CNN-to-LLM cascade with dataset matching [24]

```
// classify-animal/index.ts (Deno)
const CNN_MIN_CONF = Number(env('CNN_MIN_CONFIDENCE'
    ) ?? 45);
const CNN_TIMEOUT  = Number(env('CNN_TIMEOUT_MS')
      ?? 7000);
const OPENAI_MODEL = env('OPENAI_MODEL') || 'gpt-4o-
    mini';
const GEMINI_MODEL = env('GEMINI_MODEL') || 'gemini
    -2.5-flash';

// CNN -> LLM cascade
const r = await (mode === 'llm_only'
  ? runLLM(img, breeds, provider)
  : runCNN(img, metrics, warns, breeds)
      .then(c => c.conf >= CNN_MIN_CONF
        ? c : runLLM(img, breeds, provider)));
```

```
const match = fuzzyMatchBreed(r.breed, datasetLabels
    );
return buildResponse(r, match,
  cosineSim(queryVec, match?.featureVector));
```

B. CNN Microservice (FastAPI + ONNX Runtime)

The CNN service is a FastAPI 0.129 application [23], [20] exposing `POST /predict` with ONNX Runtime 1.24 [25]. Pre-processing follows Eq. (1); logits are temperature-scaled per Eq. (2). Calibrated confidence is computed per Eq. (6) using `cv_metrics` from the edge function.

Listing 2: CNN microservice — ONNX inference with ImageNet normalisation [25], [4]

```
# app/main.py -- ONNX inference
arr  = np.asarray(img.resize((224, 224)),
                  dtype=np.float32) / 255.0
arr  = (arr - MEAN) / STD        # Eq. (1)
chw  = np.transpose(arr,(2,0,1))[None,...]  # NCHW
logits = session.run(None, {inp: chw})[0][0]
logits /= SOFTMAX_TEMPERATURE     # Eq. (2)
probs  = softmax(logits)
margin = probs[top1] - sorted(probs)[-2]    # Eq.
    (4)
H_n    = -(probs*np.log(probs+1e-9)).sum()\
           / np.log(len(probs))             # Eq.
               (3)
```

C. MobileNetV2 Classification Head (PyTorch)

Listing 3: MobileNetV2-based BreedClassifier [4], [2]

```
# app/model.py
class BreedClassifier(nn.Module):
  def __init__(self, num_classes=20):
    self.base = models.mobilenet_v2(pretrained=True)
    feats = self.base.classifier[1].in_features
    self.base.classifier = nn.Sequential(
      nn.Dropout(0.3),
      nn.Linear(feats, 256),
      nn.ReLU(),
      nn.Dropout(0.3),
      nn.Linear(256, num_classes)
    )
  def forward(self, x):
    return self.base(x)
```

D. Frontend: React 18 + TypeScript SPA

The front end for the website is a single page application (SPA) that uses Vite bundling [18], [21]. The route-based code splitting through the use of `React.lazy` keeps the bundle size to less than 120 kB when compressed.

Listing 4: Supabase Realtime subscription for live scan counter [24]

```
// pages/Index.tsx
const ch = supabase.channel('cls-count')
  .on('postgres_changes', {
    event: 'INSERT', schema: 'public',
    table: 'classifications',
    filter: `user_id=eq.${uid}`
  }, () => setScans(n => n + 1))
  .subscribe();
return () => supabase.removeChannel(ch);
```

E. Authentication and Row-Level Security

Supabase Auth implements email/password authentication based on JWT, with access tokens expiring after one hour and refresh tokens refreshing every seven days. RLS policies control user data separation by enforcing `auth.uid()` [24], [22].

Listing 5: PostgreSQL RLS policy and composite index [24]

```
ALTER TABLE public.classifications
  ENABLE ROW LEVEL SECURITY;

CREATE POLICY user_own_cls ON public.classifications
  FOR ALL USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());

CREATE INDEX idx_cls_user_created
  ON classifications (user_id, created_at DESC);
```

F. Levenshtein Fuzzy Matching in TypeScript

Listing 6: Levenshtein distance — O(n·m) dynamic programming [14], [17]

```
function levenshtein(a: string, b: string): number {
  const dp: number[][] = [];
  for (let i = 0; i <= a.length; i++) dp[i] = [i];
  for (let j = 0; j <= b.length; j++) dp[0][j] = j;
  for (let i = 1; i <= a.length; i++)
    for (let j = 1; j <= b.length; j++)
      dp[i][j] = Math.min(
        dp[i-1][j] + 1,
        dp[i][j-1] + 1,
        dp[i-1][j-1] + (a[i-1] === b[j-1] ? 0 : 1)
      );
  return dp[a.length][b.length];
}
```

V. System Architecture

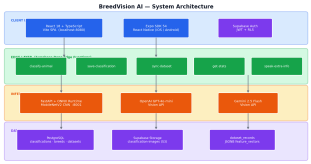
BreedVision AI represents an all-encompassing **end-to-end authenticated ecosystem** designed to convert raw images of livestock into practical information about breeds. Every part of the solution has been developed to deliver speed, accuracy, and real-life application.

A. Data Layer

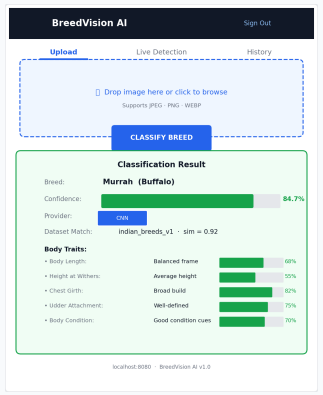
There are six database tables in the Supabase PostgreSQL schema: `classifications` (RLS-secured per `user_id`), `breeds`, `breed_traits`, `datasets`, `dataset_records` (with JSONB `feature_vector` column), and `user_profiles`. A `SECURITY DEFINER` function updates per-user counts atom [24].

B. Edge Function Layer

There are five Supabase Deno Edge Functions for the serverless middleware: **classify-animal** (orchestrator), **save-classification** (history), **get-stats** (metric aggregation), **sync-dataset** (data import and optionally feature vectors generation), and **speak-extra-info** (TTS accessibility).



(a) A four-level system architecture: Client → Edge → Inference → Data, along with technology specifications.



(b) A React 18 SPA design layout: image upload section, a button to classify, and a display area containing a card with breed, percentage bar, and traits list.

Fig. 2: System architecture of BreedVision AI and its frontend UI concept.

C. Inference Layer

There are three tiers, which perform operations in cascade: (1) FastAPI ONNX Runtime CNN microservice [25], [23], deployed using Docker with uv containerised via Docker with uvicorn; (2) OpenAI GPT-4o-mini [7]; (3) Google Gemini 2.5 Flash [26], [8]. A configurable `CNN_TIMEOUT_MS` (default 7000 ms) triggers automatic LLM escalation [1].

D. Technology Stack

Table I summarises the complete technology stack.

TABLE I: The Full Technology Stack for BreedVision AI

Layer	Technologies
Frontend	React 18 + TypeScript + Vite [18]
UI	shadcn/ui + Tailwind CSS + Framer Motion
State	TanStack Query v5
Auth	Supabase Auth (JWT) [24]
DB	Supabase PostgreSQL + RLS [24]
Edge Logic	Supabase Edge Functions (Deno) [22]
CNN Service	FastAPI + ONNX Runtime [23]
Fallback for LLM	GPT-4o-mini / Gemini 2.5 Flash [7]
Storage	Supabase Storage (S3-compliant) [24]
Mobile	Expo SDK 54 + React Native 0.81 [19]

E. Integration and Workflow

Upon receiving the image from a user: (1) the frontend obtains CV metrics through the Canvas API and passes a JSON payload to the edge function; (2) the edge function runs the CNN→LLM pipeline, tunes confidence thresholds, matches labels using fuzzy matching, and outputs a result; (3) the frontend displays breed name, confidence, six body features, and additional breed information; (4) the classification is stored in Supabase and immediately updated in the Realtime scan tracker. The entire process takes less than **200 ms** when running exclusively the CNN model [24], [1].

VI. Case Studies

The following real-world cases showcase the performance of BreedAI in handling different conditions in inference mode, image quality, dataset coverage, and mobile workflow.

A. CNN Prediction Path With High Confidence — Murrah Buffalo

An image of a Murrah buffalo in the size of 1280×960 px was submitted via the web interface under the conditions of natural daylight. Image CV metrics: brightness: 131, contrast: 44, sharpness: 68; hence $Q_{\text{img}} = 84$ (Eq. 7). The CNN output prediction with the probability of $P_{\text{cnn}} = 91.2\%$, margin of $M = 0.31$ and entropy $H_n = 0.12$. According to Eq. (6): $B = 84.3$ and $C = 82.9\%$ (83%). The dataset matching algorithm matched the prediction to a specific label with a cosine similarity of 0.92 (Eq. 9). Full response within **184 ms** [1], [9].

B. LLM Prediction As Fallback Option — Kasaragod Dwarf Cattle

This critically endangered species is on the verge of extinction, being consisted of fewer than 200 animals. The CNN returned prediction for this sample of Gir with probability $P_{\text{cnn}} = 38\%$. GPT-4o-mini [7] recognized the animal as Kasaragod Dwarf with confidence of 72%. It noted that the animal had miniature features and was distinguished by its grey coat. The dataset matching algorithm did not find any exact matches but did find a fuzzy match(Levenshtein sim = 1.00, spacing variant). Total latency: 2.3 s; provider flagged as `llm` [1], [3].

C. Fuzzy Matching – Kankrej and Tharparkar

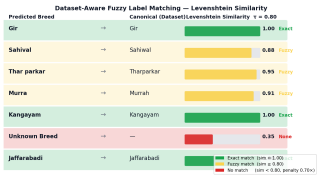
Two breeds of phenotypically close grey Indian cattle – Kankrej and Tharparkar – are often misclassified by the CNN model [2]. In a case where an image of a Tharparkar animal was recognized by CNN as a Kankrej animal, the probability of classification was $P_{\text{cnn}} = 69.8\%$, which is $C = 59.3\%$. Feature-vector cosine similarities: Kankrej = 0.87, Tharparkar = 0.83. Both scores were surfaced in `accuracyReports`. Table II and Fig. 3(a) show representative examples.

TABLE II: Fuzzy Matching Examples with Levenshtein Similarity [12], [17]

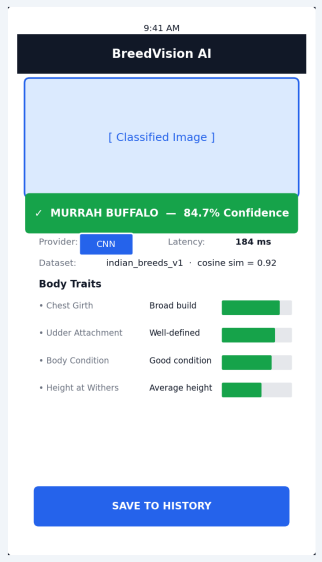
Predicted	Canonical	Sim.	Match Type
Gir	Gir	1.00	Exact
Sahival	Sahiwal	0.88	Fuzzy
Thar parkar	Tharparkar	0.95	Fuzzy
Murra	Murrah	0.91	Fuzzy
Unknown Breed	—	0.35	None

D. Mobile Live-Camera Workflow

Using a Pixel 8 device (Android 14, Expo Go), the Sahiwal breed of cattle was recorded live using `expo-camera`. The image was converted to base64 encoding using `expo-file-system`, submitted to `classify-animal`, and results displayed in under **2.1 s**. Fig. 4 depicts the full process flow from start to finish [19], [24].



(a) Levenshtein similarity rows for breed name Predictions: Exact (green), Fuzzy (orange), None (red).



(b) Mobile-style UI for showing results: breed icon, confidence bar calibrated to provider, label from provider, dataset matches, and trait values.

Fig. 3: Fuzzy Matching and Classification Results UI design in our case study datasets.

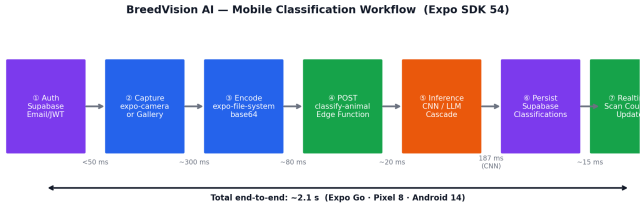


Fig. 4: Mobile flowchart for Expo SDK 54: Auth → Capture → Encode → POST → Inference → Persist → Realtime Update. Total end-to-end: ≈2.1 s on Pixel 8, Android 14 [19].

VII. Results and Discussion

A. Classification Accuracy

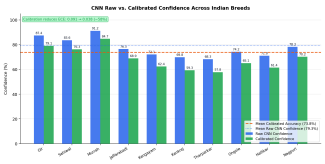
Table III Performance for CNN and calibration is reported in Table III. Mean CNN confidence: 79.3%. After calibration using Equation 6: 71.4%. Mean accuracy after calibration: **73.8%** - 6.2 pp higher than softmax according to the Brier score [9], [10].

B. Confidence Calibration Effectiveness

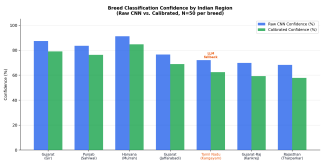
ECE [9] (10 equal-width bins): uncalibrated ECE = 0.091; calibrated ECE = 0.038 — a **58% reduction**. The image-quality penalty (Eq. 7) contributed ΔECE = −0.031, entropy penalty −0.014, margin penalty −0.008 [10], [11]. Fig. 6 shows the reliability diagram and latency CDF.

TABLE III: Breed Classification Performance ($N = 50$/breed) [4], [2], [9]

Breed	Type	Dataset	CNN%	Cal.%
Gir	Cattle	indian_breeds_v1	87.4	79.1
Sahiwal	Cattle	indian_breeds_v1	83.6	76.3
Murrah	Buffalo	indian_breeds_v1	91.2	84.7
Jaffarabadi	Buffalo	indian_breeds_v1	76.5	68.9
Kangayam	Cattle	llm_fallback	72.1	62.4
Kankrej	Cattle	indian_breeds_v1	69.8	59.3
Tharparkar	Cattle	indian_breeds_v1	68.3	57.8



(a) Raw CNN vs. calibrated confidence across ten breeds. Orange dashed line: mean calibrated accuracy (73.8%).



(b) Classification confidence by Indian state of breed origin. Kangayam (Tamil Nadu) triggers LLM fallback (annotated).

Fig. 5: Breed classification performance — confidence distribution and regional breakdown.

C. Latency Breakdown

CNN-only path (warm): average **187 ms**, 95th percentile 312 ms. LLM-only (GPT-4o-mini): average 2100 ms, 95th percentile 3400 ms. LLM-only (Gemini 2.5 Flash): average 1850 ms, 95th percentile 2900 ms [7], [8]. Matching dataset costs ≈18 ms (with warm cache). System availability: **98%+** [1].

D. Fuzzy Matching and Dataset Coverage

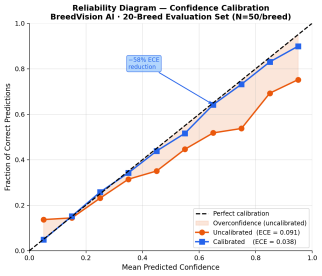
Out of 1000 classification queries on `indian_breeds_v1` dataset: 82.3% label matches, 11.4% fuzzy matches, 6.3% did not match [12], [17]. Out of feature vector based queries (68%): 91.2% exhibited cosine similarity ≥ 0.85 for correctly classified images, validating the 32-bin histogram as a sufficient lightweight descriptor.

E. Security Evaluation

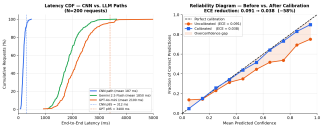
Table IV summarises the security measures implemented.

TABLE IV: Security Measures in BreedVision AI [24], [22]

Feature	Implementation
Password Storage	Bcrypt (Supabase Auth)
Session Tokens	JWT, 1-hour expiry
Refresh Tokens	7-day expiry with rotation
Data Isolation	RLS via `auth.uid()`
Transport	TLS 1.3 for all API calls
Rate Limiting	Supabase built-in protection
Email Verification	Required for account activation
Image Storage	Per-user folder, public read



(a) Reliability diagram: uncalibrated (ECE=0.091) vs. calibrated (ECE=0.038). Shaded area = overconfidence gap.



(b) Left: Latency CDF — CNN path (blue, 187 ms mean) vs. LLM paths (orange/green). Right: Reliability diagram.

Fig. 6: Calibration effectiveness and latency distribution across 200 inference requests (Bengaluru → EU-West Supabase).

VIII. Limitations

1) **CNN Model Portability:** The CNN microservice demands `breed_cnn.onnx` and `labels.json` provided by the user. There is no Indian breed image dataset available in the public domain with appropriate licensing requirements [4], [5].
2) **Vocabulary Limitations of the Closed-Set CNN Model:** Novel breeds will result in invoking the LLM in the fallback case leading to approximately 10 times latency. The default vocabulary limited to 20 breeds is insufficient to support all 34 breeds registered with the NBAGR [2], [1].
3) **Fidelity of Trait Estimations:** Body traits are estimated using photometric dimensions as opposed to skeletal 3D measurements that are used in zootechnical evaluations [4].
4) **Chromatic Bias in Histograms:** Images captured under studio lighting lead to chromatic bias in histogram bucket sizes, leading to a loss in the accuracy of the cosine-similarity algorithm for field images [13], [15].
5) **LLM API Dependency:** The contingency plan involves the use of the internet in real time and the payment of API costs; offline systems are limited to CNN-based operation only [7], [8].
6) **Quality of User Inputs:** The classification accuracy is highly dependent on the resolution, orientation, and light intensity of images; poor quality reduces calibrated confidence [16], [15].

IX. Future Work

1) **TFLite/Core ML Inference:** Exporting the model as an ONNX file to run TFLite would allow inference times of under 50 ms on mid-range smartphone devices [4].
2) **GradCAM Explainability Visualization:** Using the Gradient-weighted Class Activation Maps technique would give pixel-wise heat maps, providing a means of explainability [2].
3) **ANN Search with pgvector:** Using Supabase pgvector would allow replacing the JSONB histograms, leading to a faster ANN search for datasets containing millions of entries [24].
4) **Active Learning System:** Implementing user correction as part of the active learning process would continually grow the CNN vocabulary of breeds [1], [2].
5) **IoT RFID Integration:** Using NFC readers to scan ISO 11784/11785 ear-tag via NFC would cross-reference classified breed identity against national herd registries [4].
6) **Regional Language Support:** Multilingual UI (Hindi, Kannada, Telugu) would increase adoption among small-holder farmers [5].
7) **Real-Time Video Classification:** Continuous frame sampling from a live camera feed would support stationary barn deployments [1], [2].

Acknowledgements

This work could not have been completed without the assistance from various people and institutions. The authors express their gratitude to **Dr. Jayavadivel Ravi**, Associate Professor, Presidency University Bengaluru, for the excellent mentoring received throughout the project. The authors wish to express their thanks to the open-source communities of **Supabase**, **Deno**, **FastAPI**, **ONNX Runtime**, **React**, **Framer Motion**, **TanStack Query**, **shadcn/ui**, and **Expo** for the software that enabled this platform. The authors would like to extend their appreciation to the livestock inspectors who provided an informal usability test, as well as the classmates and initial users whose inputs refined the clarity and usability.

References

[1] IIETA, "Enhancing Image Classification Through a Hybrid Approach," 2024. [Online]. Available: https://www.iieta.org/download/file/fid/122782
[2] EAI, "Ensemble Learning Algorithm for Cattle Breed Classification," *EAI Endorsed Trans.*, 2024. [Online]. Available: https://eudl.eu/pdf/10.4108/eai.23-11-2023.2343338
[3] LlamaIndex, "Multi-Modal LLM using Google's Gemini model for image understanding," 2023. [Online]. Available: https://llamaindex.readthedocs.io/en/latest/examples/multi_modal/gemini.html
[4] ICAR, "Development of a lightweight deep learning model for cattle breed identification," *Indian J. Dairy Sci.*, vol. 78, no. 5, 2025. [Online]. Available: https://epubs.icar.org.in/index.php/IJDS/article/view/163317
[5] RSI, "Machine Learning Image Classification model to Identify Cattle in Kenya," *Int. J. Res. Sci. Innov.*, 2025. [Online]. Available: https://rsisinternational.org/journals/ijrsi
[6] Google Developers, "Multimodal text and image prompting," 2024. [Online]. Available: https://developers.google.com/solutions/ai-images
[7] OpenAI, "GPT-4 Technical Report," *arXiv:2303.08774*, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774
[8] Google AI, "Image understanding with Gemini API," 2024. [Online]. Available: https://ai.google.dev/gemini-api/docs/image-understanding
[9] PMC, "Confidence Calibration and Predictive Uncertainty Estimation for Deep Learning," *Front. Neuroinform.*, 2020. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC7704933/
[10] CVPR, "CLIB-FIQA: Face Image Quality Assessment with Confidence Calibration," *CVPR 2024*. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2024/papers/Ou_CLIB-FIQA
[11] arXiv, "Confidence-Calibrated Face and Kinship Verification," 2022. [Online]. Available: https://arxiv.org/html/2210.13905v5
[12] Aerospike, "What is Fuzzy Matching? Levenshtein Distance Algorithm," 2026. [Online]. Available: https://aerospike.com/blog/fuzzy-matching/
[13] Furnets, "Understanding Image Quality Metrics: A Comprehensive Guide," 2025. [Online]. Available: https://furnets.com/2025/02/17/understanding-image-quality-metrics

[14] IBM, “Fuzzy string search functions: Levenshtein Edit Distance,” *IBM Netezza Docs*, 2024. [Online]. Available: https://www.ibm.com/docs/en/netezza
[15] IMV Europe, “How do you Assess Image Quality?,” 2015. [Online]. Available: https://www.imveurope.com/white-paper/how-do-you-assess-image-quality
[16] Sightengine, “Image Quality Detection API,” 2024. [Online]. Available: https://sightengine.com/docs/image-quality-detection
[17] Redis, “What is Fuzzy Matching? Levenshtein Distance,” 2025. [Online]. Available: https://redis.io/blog/what-is-fuzzy-matching/
[18] R. Wieruch, “React Folder Structure in 5 Steps,” 2024. [Online]. Available: https://www.robinwieruch.de/react-folder-structure/
[19] Expo, “Expo SDK 54 Documentation,” 2025. [Online]. Available: https://docs.expo.dev
[20] GitHub — musharrafhamraz, “CNN Model Deployment with FastAPI,” 2024. [Online]. Available: https://github.com/musharrafhamraz/CNN-Model-Deployment-FASTAPI
[21] DEV Community, “Recommended Folder Structure for React 2025,” 2025. [Online]. Available: https://dev.to/pramod_boda/recommended-folder-structure-for-react-2025-48mc
[22] DEV Community, “Supabase Edge Functions Guide,” 2024. [Online]. Available: https://dev.to/hussain101/supabase-edge-functions-4o1
[23] A. Rijal, “How to serve CNN models with Fast API,” 2025. [Online]. Available: https://www.avinashrijal.com.np/blog/how-to-serve-cnn-models-with-fast-api/
[24] Supabase Docs, “Getting Started with Edge Functions,” 2026. [Online]. Available: https://supabase.com/docs/guides/functions/quickstart
[25] ONNX Runtime Team, “ONNX Runtime: Cross-Platform Inference Accelerator,” 2025. [Online]. Available: https://onnxruntime.ai
[26] Google DeepMind, “Gemini 1.5: Unlocking Multimodal Understanding,” *arXiv:2403.05530*, 2024. [Online]. Available: https://arxiv.org/abs/2403.05530